COL761/COL7361/AIL7026
Data Mining
Assignment 1 Report

Task 1: Frequent Itemset Mining
Apriori vs FP-Growth Comparison

---

1 Introduction

Frequent itemset mining aims to discover patterns in transactional datasets. We evaluate two classical algorithms: **Apriori** (candidate-based) and **FP-Growth** (tree-based). This report compares their empirical runtime performance across different support thresholds on both real-world and synthetic datasets.

2 Implementation Details

To address the experimental reproducibility, the following environment and procedures were utilized:

- **Software Environment:** Experiments were conducted on a macOS-based system using `Python 3.13.3`. Plotting was performed using `matplotlib` with a logarithmic scale for the Y-axis to visualize the orders-of-magnitude difference in runtimes.
- **Compilation:** The provided C source code for Apriori and FP-Growth (Christian Borgelt's implementations) was compiled using the included `Makefiles` via the `gcc` compiler. Optimization flags in the Makefiles were left at default.
- **Automation:** Bash scripts (`q1_1.sh` and `q1_2.sh`) were developed to automate the compilation, mining, and plotting pipelines. These scripts utilize the `timeout` utility to prevent execution from exceeding 3600 seconds per threshold.

3 Runtime Comparison on Original Dataset

The runtime comparison on the `webdocs.dat` dataset is shown in Figure 1.

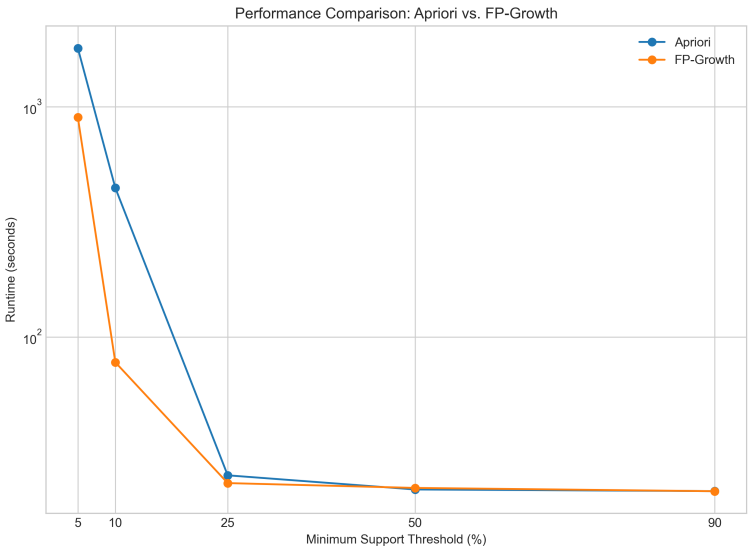

Figure 1: Runtime comparison on original dataset (`webdocs.dat`).

3.1 Observations & Analysis

1. **Apriori Complexity:** At low supports (5%–10%), Apriori generates millions of candidate itemsets. The overhead of multiple database scans and subset checking causes runtimes to exceed 1000 seconds.

2. **FP-Growth Efficiency:** FP-Growth remains significantly faster (often 10-50x) by compressing the data into an FP-Tree, requiring only two passes over the data.

3. **Convergence:** At high support thresholds (50% and 90%), the number of frequent itemsets drops to near zero. Both algorithms converge to a baseline runtime of <1 second as the search space becomes trivial.

4 Runtime Comparison on Generated Dataset

For Task 2, we constructed a synthetic dataset specifically designed to stress-test the algorithms and replicate the "dense core" behavior seen in Figure 1 of the assignment handout.

Figure 2: Runtime comparison on Generated Dataset (`generated_transactions.dat`).

4.1 Dataset Generation Strategy

The dataset was generated using a **Dense Core-Background model** with the following logic:

- **Core Density:** A subset of 35 items was chosen to form a core. These items appear in 88% of all transactions. To maximize density, core transactions contain all 35 items.
- **Combinatorial Explosion:** By setting the core support to 88%, these 35 items remain frequent at the 5%, 10%, 25%, and 50% thresholds. Apriori is thus forced to explore a search space of 2^{35} potential candidates, leading to a consistent timeout (flat high line).
- **Threshold Drop:** At the 90% threshold, the core items (88% support) are no longer frequent. The search space collapses instantly, causing the sharp drop in runtime seen in Figure 2.

5 Comparison: Original vs. Generated

The generated dataset exhibits a more extreme performance gap than the original dataset. While `webdocs` is naturally sparse, the synthetic dataset creates a "perfect storm" for Apriori by providing a large, frequent, and perfectly dense core. This amplifies the candidate explosion effect while allowing FP-Growth to demonstrate its superior compression through a very deep but narrow FP-Tree.

6 How to Run

6.1 Task 1.1: Original Dataset Pipeline

```
# Execute mining and plotting for webdocs.dat
bash q1_1.sh "./apriori/apriori/src/apriori" \
             "./fpgrowth/fpgrowth/src/fpgrowth" \
             "webdocs.dat" \
             "./output_task1_1"
```

6.2 Task 1.2: Synthetic Dataset Pipeline

Note: The updated `q1_2.sh` script now automatically triggers the mining and plotting pipeline after dataset generation to ensure full automation.

```
# Generate dataset AND run the mining/plotting pipeline
bash q1_2.sh "1-300" 15000
```